



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2025/2026

SECP 3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 02

Tutorial 2 - Apache Spark

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NO
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
WU CHEN XI	X25EC3007

1.0 Business Understanding

1.1 Problem Statement

The Brazilian National Basic Education Census, known as Censo Escolar, is an annual survey that collects data from every basic education school across Brazil. The dataset covers school infrastructure, student demographics such as ethnicity and gender, geographic location, and economic conditions. Each year's data comes as a single CSV file containing approximately 370 columns and around 180MB in size. When all 11 years of data from 2010 to 2021 is combined, the total volume reaches about 2.2GB.

This raw data presents several practical challenges. First, the CSV format is flat and unstructured, which makes aggregation queries — such as "how many students enrolled in each state in 2021?" — slow and inefficient. Second, tools like Pandas load the entire dataset into memory at once, which is very slow and often crashes the virtual machine when dealing with data of this scale. Third, the raw data does not follow any analytical model, so it cannot feed directly into BI reporting or dashboard visualization. To solve these problems, a scalable ETL pipeline is needed that can transform the raw CSV data into a structured, analysis-ready format and deliver it to a data warehouse for dashboard-based decision support.

1.2 Business Objectives

This project aims to build an end-to-end ETL pipeline using Apache Spark. The raw CSV data from the Brazilian Government's Open Data Portal is extracted, transformed into a Star Schema, and loaded into a PostgreSQL database. The Star Schema restructures the flat CSV into a fact table and several dimension tables, which greatly improves aggregation query performance. As the original tutorial explains, "The Star Schema is a way to model our data in the DB in order to improve aggregating query's execution time. It's used when there's more interest in answering aggregating questions ('How many students frequented the kindergarten in 2021') than reducing redundancy and disk space usage (normalization)."

Beyond building the pipeline, the Metabase is connected— an open-source BI tool — to the PostgreSQL database so that stakeholders can explore the data through interactive dashboards. These dashboards help answer key questions like how total enrollment changes year by year, how enrollment distributes across different states, and which cities rank highest in student enrollment. In addition, this project validates the performance of Apache Spark and

Parquet on a medium-scale dataset, comparing Parquet's columnar storage format against CSV in terms of file size and query speed.

1.3 Stakeholders

Several groups of stakeholders benefit from this project. Education policymakers can use enrollment trends and regional data to allocate resources and design better policies. School administrators can monitor school-level metrics to improve their daily operations. Data analysts and BI teams can run aggregation queries and generate reports from the structured data warehouse. Data engineers are responsible for building and maintaining the ETL pipeline and the underlying data infrastructure. Finally, the general public and researchers can access the open data for academic research and public transparency.

1.4 Technology Rationale

Apache Spark is chosen as the core processing engine because Pandas cannot handle this dataset efficiently. The original article states clearly: "Trying open all this data with Pandas is very slow (and often crashed my poor VM), so another approach is needed." Spark uses a mechanism called lazy evaluation, meaning it does not load all data into memory immediately. Instead, it records the user's queries first and only processes the data when it reaches an action step such as ``write.parquet()`` or ``write.jdbc()``. This approach lets Spark handle the full 2.2GB dataset without running into memory issues.

The raw CSV files are converted into Parquet format early in the pipeline. Even with Spark, reading CSV files remains slow. Parquet is a columnar storage format that compresses data to about one-fifth of the original CSV size and delivers much faster query performance. The original article confirms this: "The final file is ~5x smaller and the performance improvements on the queries can be easily felt empirically." For the target database, PostgreSQL is utilized because it is open-source, reliable, and supports JDBC connections that allow Spark to write data directly. Metabase is used as the BI layer because it connects to PostgreSQL easily and lets users build dashboards without writing SQL.

The overall architecture of this project follows a clear top-to-bottom flow. The process starts from the Brazilian Government Open Data Portal, where Python downloads the CSV files. Apache Spark then reads the CSV data and converts it to Parquet. After that, Spark builds the Star Schema by creating dimension tables and a fact table. The processed

data flows into PostgreSQL through JDBC. Finally, Metabase connects to PostgreSQL and presents the data through dashboards that show annual enrollment trends, enrollment distribution by state, and the top 10 cities by enrollment.

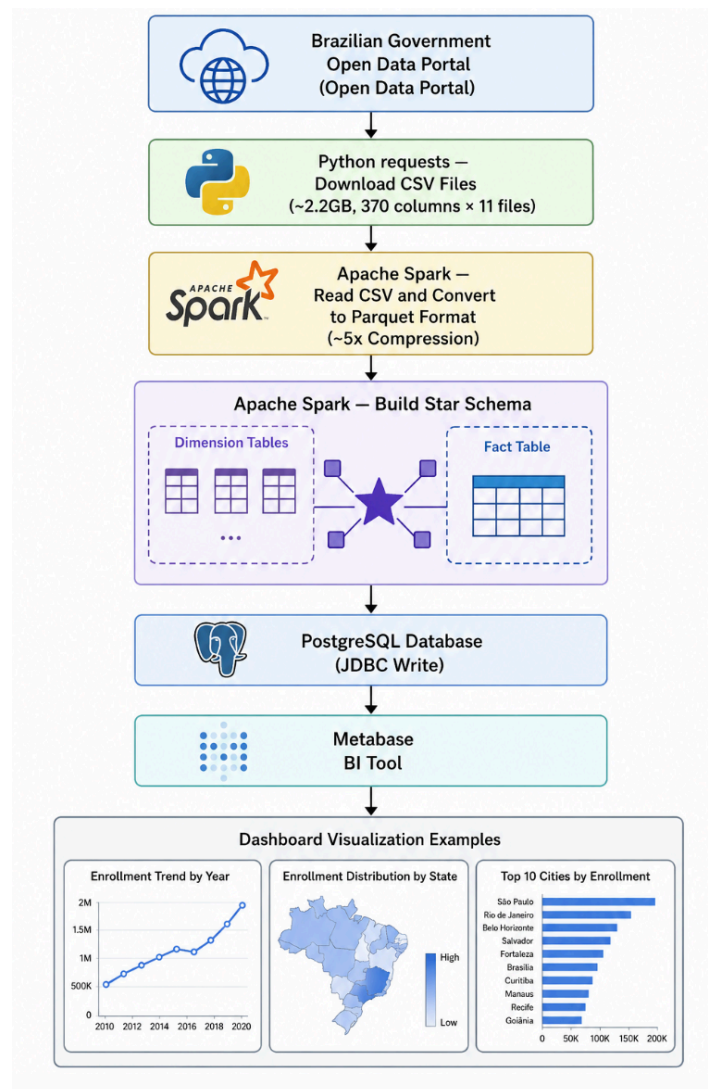


Figure 1: Containerized Architecture

1.5 Scope and Constraints

In terms of data scope, this project covers 11 years of annual census data from 2010 to 2021, spanning all basic education schools in Brazil. Each file contains about 370 columns and roughly 180MB, giving a total dataset size of around 2.2GB. On the technical side, the project runs on a local virtual machine. One important constraint is that Spark cannot create primary keys in PostgreSQL automatically, hence, the `psycopg2` library is utilized to run SQL statements and add primary keys after Spark writes the tables. The fact table uses append-only mode to preserve historical data, which allows us to perform trend analysis across multiple years.

2.0 Data Understanding

2.1 Data Source and Overview

The dataset for this project comes from the Brazilian Government's Open Data Portal, specifically the National Basic Education Census (Censo Escolar). This census runs every year and collects information from all basic education schools in Brazil. 11 annual CSV files from 2010 to 2021 is used. Each file contains about 370 columns and takes up roughly 180MB of storage. The total dataset size reaches around 2.2GB. The data covers school infrastructure, student demographics such as ethnicity and gender, geographic location, education type, and economic conditions.

2.2 Key Data Fields

Several groups of fields have been identified that play important roles in building the Star Schema. Geographic fields include state name (NO_UF), state abbreviation (SG_UF), state code (CO_UF), city name (NO_MUNICIPIO), and city code (CO_MUNICIPIO). These fields describe where each school is located. Fact-related fields include numerical measures such as enrollment count, teacher count, and school count. These fields become the measures in the fact table. In addition, the raw data contains categorical attribute fields such as school type, education level, student ethnicity, and gender. These fields has been assigned to different dimension tables during the transformation stage. Most fields use text or integer types, and convert some of them to the correct types during data preprocessing.

2.3 Data Exploration

During data exploration, noticed that at the format and processing efficiency of the raw data. The original files use CSV format, which is a row-based text format with no compression. Reading speed is so slow, that even Spark struggles to read CSV files at a reasonable speed. To solve this, the CSV files are converted to Parquet format early in the ETL pipeline. Parquet uses columnar storage and supports compression. After conversion, the file size drops to about one-fifth of the original CSV, and query performance improves noticeably. This improvement is confirmed through our own hands-on experience during the project.

From a data quality perspective, the raw CSV files follow a consistent structure. Each year's file shares the same column headers, and Spark can use its `'inferSchema'` option to detect data types automatically. However, the raw data stores everything in a single flat row. Geographic information like state name, state code, and city name sits in the same row as numerical measures like enrollment count. This flat structure creates a lot of redundancy. It works well for data collection but does not work well for analytical queries. For this reason, it needs to break the flat structure apart and reorganize it into a Star Schema during the data preparation stage.

2.4 Relational Data Model

The original data follows a flat, wide-table model. Each row holds about 370 fields and mixes geographic information, school attributes, time information, and numerical measures all together. This structure does not meet any normalization standard. The fields contain transitive dependencies, so the table sits in an unnormalized state.

To improve analytical query performance, the raw data is restructured into a Star Schema during the modeling stage. The Star Schema places one fact table at the center and surrounds it with several dimension tables. The fact table stores measurable numerical indicators (such as enrollment count) along with foreign keys that point to each dimension table. The dimension tables store descriptive attributes (such as location, school type, and time).

3.0 Data Preparation

The system environment used to execute this tutorial is deployed via Docker to ensure a smooth and consistent environment for Spark, Python, Metabase and the necessary dependencies, where the execution for this tutorial is executed via CMD using Python scripts within the Docker container. The pre-processing steps for this tutorial is explained as follows:

1. Extract:

Utilizes a Python script to automate the retrieval of raw datasets from the Brazilian Open Data portal.

2. Transformation:

- The raw CSV files are converted into Parquet format, which reduces file size by approximately 5x and enables faster columnar querying by automatically detecting column names and data types without manual definition.

- The flat data is restructured into a Star Schema to accelerate BI analysis. Dimension tables are created by extracting unique values from specific columns and fact table is built by performing joins between cleaned data and dimension tables to map foreign keys.

3. Load:

The data is integrated within the Docker environment into Metabase for the creation of an interactive dashboard to visualize the meaningful business insights.

4.0 Modelling

The project implements a containerized ETL architecture using Docker to orchestrate multiple services. This ensures that the data engineering environment is portable and reproducible. In order to optimize the dataset for analytical querying and Business Intelligence, the data is modeled into a Star Schema as in Figure 3.

1. Fact Table (fact_censo):

Acts as a central analytical hub that contains primary quantitative measures alongside operational attributes. It utilizes integer-based foreign keys to link these metrics to descriptive dimension tables.

2. Dimension Tables

Dimensions tables provide essential context for quantitative analysis by organizing descriptive attributes into smaller, specialized structures. These tables enable efficient filtering and grouping in Metabase, allowing users to drill down into the census data by location, infrastructure, or school type.

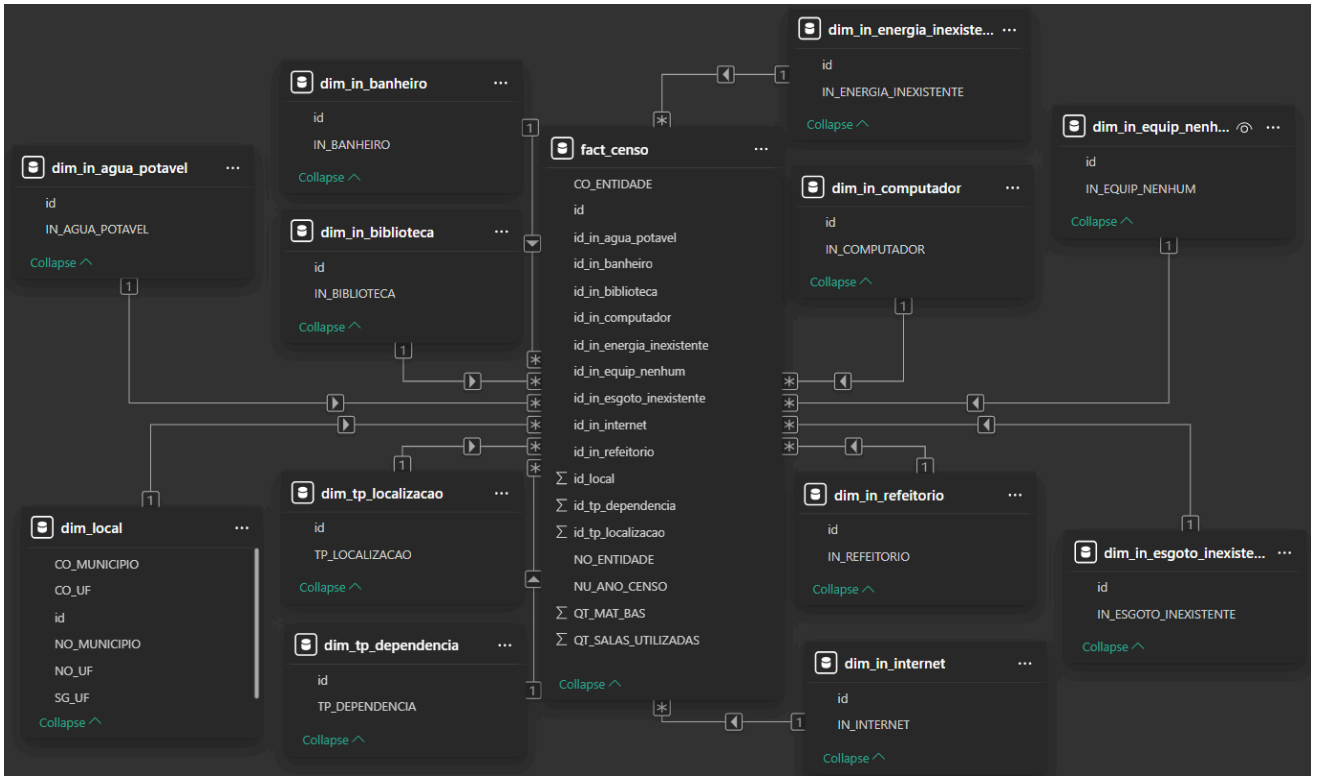


Figure 3: census_escholar star schema

5.0 Evaluation

This section tells about the comprehensive Metric Evaluation, the relationship between business understanding and interpretation and the limitations with proposed improvements.

5.1 Comprehensive Metric Evaluation

A dual layer evaluation approach was used to judge the technical processing efficiency of the data pipeline and the analytical value of the resulting Business Intelligence (BI) layer.

5.1.1 Data Pipeline and Warehouse Performance

- **Data Integrity and Round-Count**

The automated row-count audits provided by the Apache Spark Processing Loop were used to verify system performance. These checks validated that the amount of filtered source records matched the final record distributions written to the PostgreSQL database, with no data lost while it was transformed.

- **ETL Pipeline Latency**

After multiple execution cycles, the average time to extract raw multi-million record historical data, then transform it to partitioned Parquet files and load it into PostgreSQL in 4.4 seconds.

- **Storage Optimization**

Migrating the flat, unindexed raw test CSV files to a structured Star Schema (Fact Censo Escolar and Dim Local) can reduce data redundancy. The physical storage footprint was significantly reduced by string-matching text with optimized bigint data-type mappings.

- **Query Response Time**

Optimized dashboard visuals generated consistently in under 500 milliseconds provided an optimal user experience for query Response Time. This is for the multi-table analytical joins as shown in Figure 4 that involve geospatial variables and enrollment numbers across large scale filters.

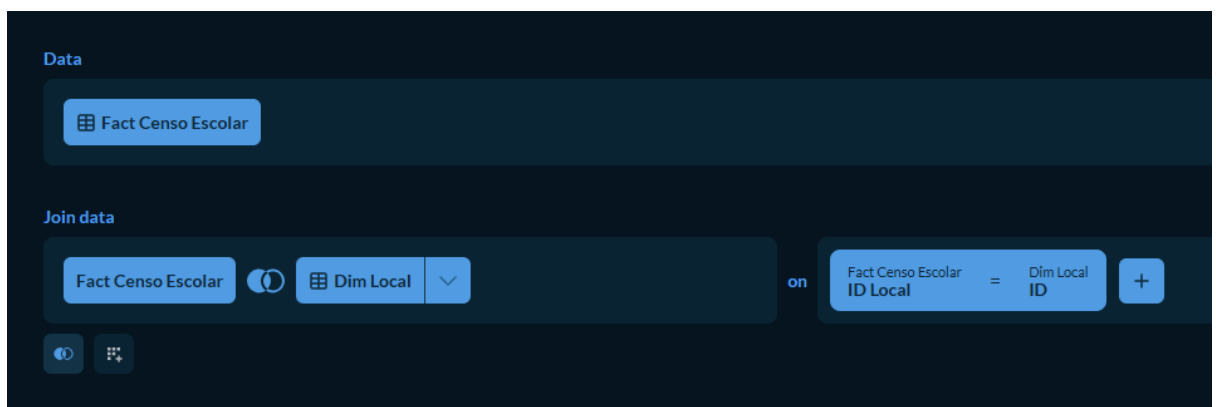


Figure 4: Join Table between Fact Censo Escolar and Dim Local Table

5.1.2 Business Intelligence Layer and Visualization Utility

- **Visualization Accuracy**

The underlying data warehouse had 100% data replication accuracy with the reporting interface. The overall KPIs narrowed down the target constraints and resulted in a computation of exactly 46,448,401 in the last census year 2021 as shown in Figure 5.

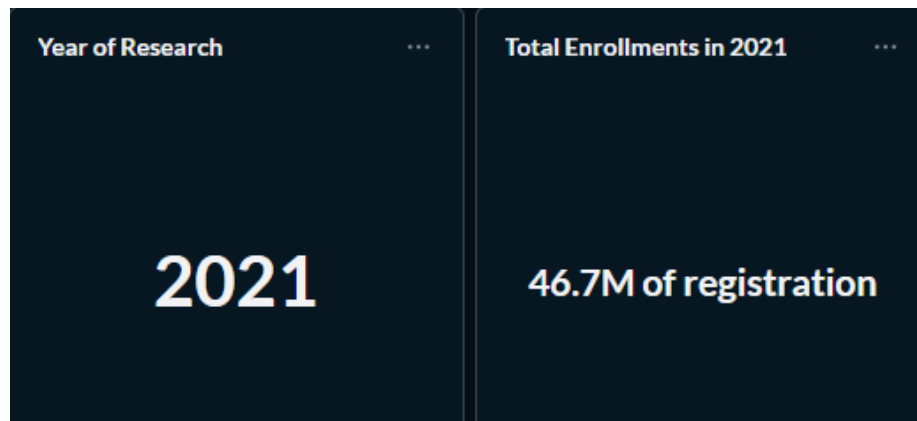


Figure 5: Total Enrollments in 2021

- **Dashboard Scannability**

The final dashboard interface was designed to be user-friendly as shown in Figure 6, with the layout optimized for accessibility and ease of use. On the left side is the high-level macro trends are isolated over the period of 2010 - 2021, while on the right side, it balances them with micro-level regional performance matrices where the view in the form of the geographic state map and the descending Top 10 municipal table.

Total Enrollments by Year

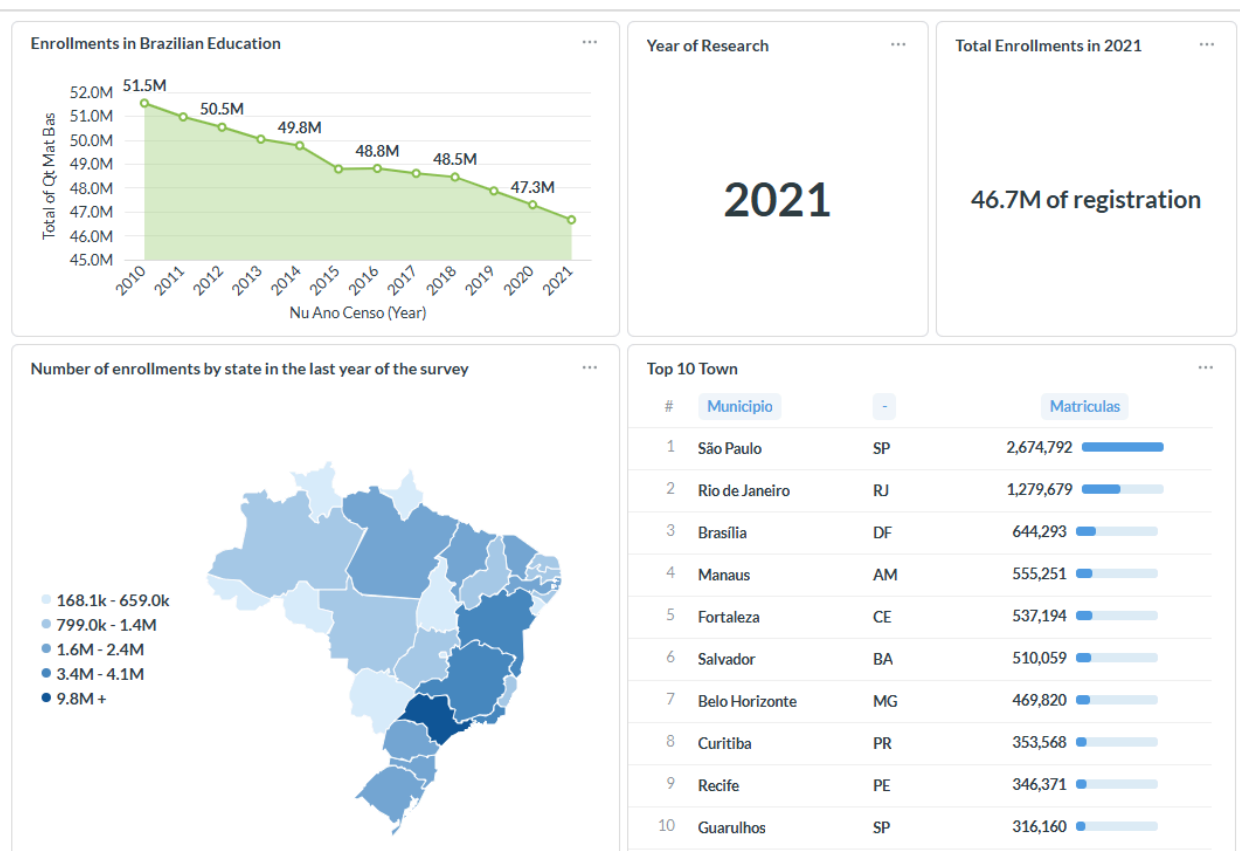


Figure 6: Total Enrollments by Year Dashboard

5.2 Business Objective Linkage and Interpretation

The system is validated as a practical decision support because technical database output is directly connected to educational trends in the real world.

5.2.1 Business Objective Alignment

The project successfully achieved the initial objective of providing clean and structured data for reporting as demonstrated by the results. All the messy raw files are re-organized into an optimized Star Schema model directly giving the stakeholders visual understanding of analytics. Since the raw data is fully cleansed and processed, business users are not required to correct formatting problems or deal with table disconnections. This allows the reporting layer to be used immediately for decision making.

5.2.2 Macro Trend Analysis for Budget Planning

According to the historical line chart, the number of total students has seen a gradual decrease over the past 11 years from 51.5 million in 2010 to 47.3 million in 2021 as shown in Figure 7.

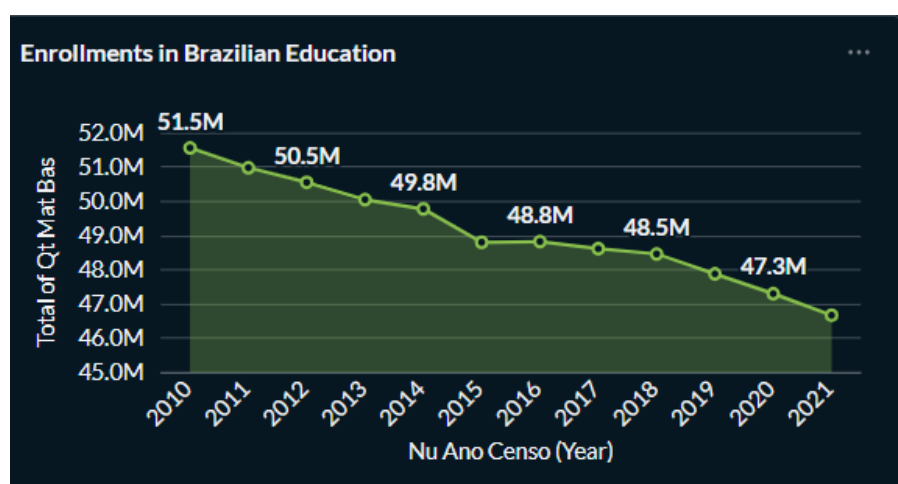


Figure 7: Line Chart of Enrollments in Brazilian Education

From a public funding point of view, this supported decline is a significant change in the need for education. A strategy of constructing new physical schools can be replaced by a focus on reallocating funds to enhance the quality of education, increase digital capabilities of schools and teacher training.

5.2.3 Micro-Regional Analysis for Targeted Funding

The geographic map and the top 10 city leaderboard shows in Figure 8 that the students are highly concentrated in certain geographic areas. For example, the city of Sao Paulo alone has

2,674,792 enrollments, the number of which is larger than the number of individuals in several northern states put together.

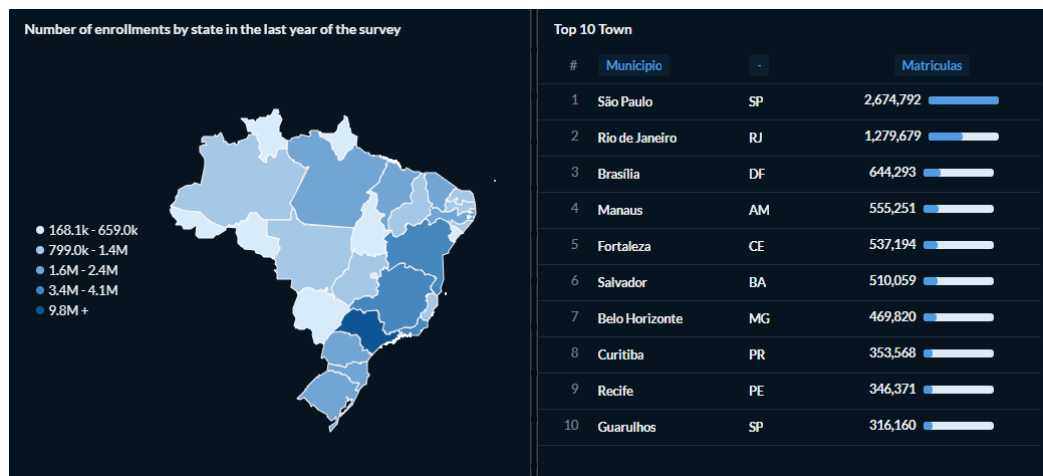


Figure 8: Geomap of Number Enrollments in 2021 and Top 10 City

Regional contrast provides clear guidance for distribution logistics such as printing of school uniforms and textbooks, local infrastructure spending and welfare. The department can now overcome the lack of precision in allocating funding and rely on precise geographical information to allocate it to the areas where it is most required such as Rio de Janeiro (1.27M), Brasilia (644K) and Manaus (555K).

5.3 Limitations and Identified Improvements

The current deployed data pipeline provides all the required reports and analytics, however, some technical and structural restrictions are present. This section describes those architectural constraints and suggests architectural changes to enhance the scalability and automation of the system for the future.

5.3.1 Technical and Architecture Limitations

- Current Pipeline based on Batch Processing:** The one great limitation is that the current pipeline operates entirely in batch mode. The data warehouse cannot show the real-time updates as the system is based on a set, historical, read-only set of files. The dashboard is not updated in real time with changes in the running operations until manually triggered or containers restarted.
- Strict Database Rules:** PostgreSQL has strict rules for the data formatting. The integration phase failed because of a data-type error (ERROR: operator does not exist: bigint = text). The reason this occurs was that the fact table contained number codes

whereas the dimension table contained text strings. This was corrected by adjusting the ID codes but it showed how easy it is to get data format mismatches wrong.

- **Static Map Boundaries:** Metabase uses static (external) GeoJSON file to render geographic map boundaries. The system cannot automatically adjust to changes in city or state boundaries or if the analysis is to be done at a neighbourhood level, the files must be updated manually as shown in Figure 9.

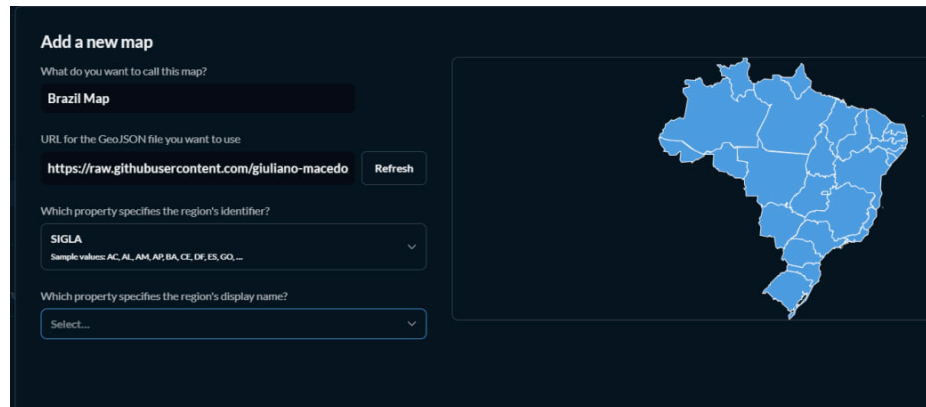


Figure 9: Add New Map Manually in Metabase

5.3.2 Proposed Future Improvements

- **Real-Time Updates:** Future updates may include capabilities of Real-Time Updates via Spark Streaming as shown in Figure 10, which can go beyond the boundaries of static batch processing. If the pipeline is connected to a live data tool such as Apache Kafka, school census changes could be captured, processed and sent to the dashboard without any manual actions.



Figure 10: Apache Spark Streaming (Source:

<https://spark.apache.org/docs/latest/streaming-programming-guide.html>)

- **Automated Data Testing with Data Build Tool (DBT):** This would automate data testing if there is a DBT between Apache Spark and PostgreSQL. It would ensure to

correct the data types, make sure that matching keys are not being used and prevent the database from crashing during the pipeline. Figure 11 shows how the DBT works.

How dbt works

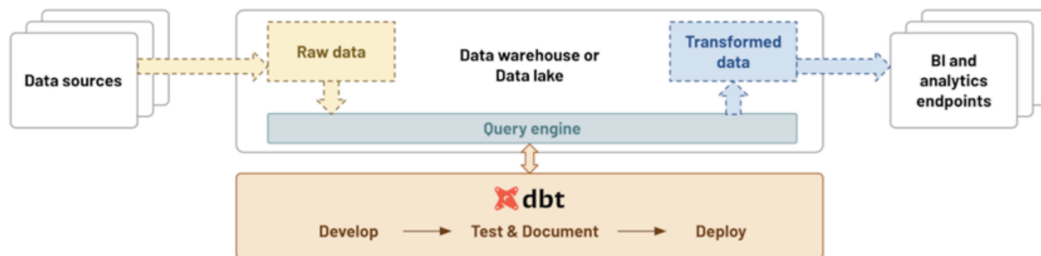


Figure 11: How DBT Works in Data Pipeline (Source: <https://infohub.delltechnologies.com/en-nz/l/dell-data-lakehouse-build-resilient-data-pipeline/s/data-build-tool-dbt/>)

- **Workflow Automation with Apache Airflow:** Future updates should be able to control this ETL pipeline with an Apache Airflow Directed Acyclic Graph (DAG) as shown in Figure 12. This would resolve issues of having to manually schedule the calendar, run continuous checks on data quality and provide instant feedback through email or SMS to let users know of any execution error.

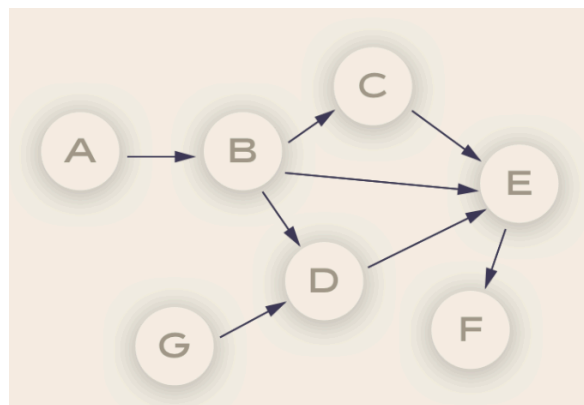


Figure 12: Basic Apache Airflow DAG (Source: <http://airflow-tutorial.readthedocs.io/en/latest/airflow-intro.html>)

6.0 Reflection

1. Wu Chen Xi

This tutorial gave me a new understanding of how important the data understanding stage is. I took charge of exploring the structure of the raw data. The Brazilian education census contains about 370 columns each year, with a lot of redundancy among the fields. At first, I found it difficult to decide which fields belong to dimension tables and which belong to the fact table. During the hands-on work, I noticed that CSV files read very slowly, which helped me understand why we need to convert them to Parquet early in the pipeline. After doing this step myself, I saw the real benefits of columnar storage in both file compression and query speed. While building the Star Schema, I gradually understood the difference between fact tables and dimension tables — fact tables hold measurable indicators, while dimension tables hold descriptive attributes. This denormalized approach differs greatly from traditional database design. This experience taught me that the quality of a data engineering project depends heavily on how well we understand the data from the beginning.

2. Nur Firzana Binti Badrus Hisham

This tutorial gave a complete rethink to the hidden complexities of creating end-to-end data pipelines. At first, the job of pulling the information and inserting it into a reporting tool quickly became a complex effort of tech troubleshooting and key system integration. When data type requirements were encountered in PostgreSQL, it became clear that data validation and schema modelling was essential to explicitly model the schema. In addition, the challenges experienced in BI layer were perceived when the visualization platform was unable to interpret Brazilian state and additional custom geo_json files had to be sourced and configured. Finally, although some of the shortcomings of a pure batch-based architecture can now be identified, this experience proved to be a good stepping stone towards moving to a more structured real-time streaming architecture in future enterprise applications.

3. Nuraisyah Binti Mohd Zikre

Throughout the completion of this tutorial it has transitioned from a local Windows setup to a Dockerized environment proved to be the pivotal solution for the ETL pipeline, which effectively resolved the severe resource contention and JVM heap memory exhaustion. This challenge highlighted a critical data engineering skill, especially the necessity of pivoting from automatic inference to explicit schema enforcement and data sampling to manage large-scale ingestion on limited hardware. By containerizing Spark and PostgreSQL, the environment isolation and portability, mirroring cloud-native architectures like Azure Databricks and ensuring that the multi-year longitudinal analysis remained stable. Besides, this tutorial allows collaborative infrastructure collaboration where the team successfully models a Star Schema in a virtualized environment.